Decentralized Intelligence for Crop Health: Optimized Federated Learning Models for Plant Disease Detection

Jaspal Kaur Saini[a], Lovepreet Sharma[b]

[a]*Department of Information Technology, Sports and Health Research Centre, Dr. B. R. Ambedkar National Institute of Technology, Jalandhar, Punjab, India*
[b]*Department of Information Technology, Dr. B. R. Ambedkar National Institute of Technology, Jalandhar, Punjab, India*

Abstract

Deep learning-based plant disease diagnosis has been identified as an efficient technique to improve crop health monitoring in smart agriculture. But traditional centralized learning methods involve aggregating private farm-level data, which poses challenges regarding data privacy, communication overhead, and scalability. Federated Learning (FL) is a decentralized learning paradigm that allows multiple clients to jointly train a shared model while maintaining data locality. In this paper, we conduct a comparative analysis of federated learning algorithms for plant disease diagnosis, considering the convergence properties and robustness of FL algorithms for non-IID (Independent and Identically Distributed) data.

We compared the performance of five state-of-the-art FL algorithms, namely FedAvg, FedProx, SCAFFOLD, FedMA, and FedOpt, on various convolutional neural network architectures for image-based plant disease classification tasks. The training dataset is split across clients to simulate practical non-IID data distributions, as typically found in geographically distributed agricultural settings. The performance of the models is measured using accuracy, precision, recall, F1-score, convergence rate, and communication overhead.

Keywords: Federated Learning, Plant Disease Detection, Machine Learning, Artificial Intelligence, Smart Agriculture

Email addresses: sainijk@nitj.ac.in (Jaspal Kaur Saini), lovepreetsharma828sharma@gmail.com (Lovepreet Sharma)

1. Introduction

Agriculture is an essential component of providing an adequate amount of food on a global level. However, there are many threats to crop productivity, including the presence of diseases on plants that, if undetected and unaddressed in a timely manner, can lead to a substantial decrease in output. Traditional methods for identifying diseases in crops are primarily based on manual inspection of the plants by trained professionals. This method is not optimal for large-scale agricultural situations due to the large amount of time it takes, the subjectivity of the process, and the impracticality of performing these inspections in a reasonable amount of time.

With the advent of computer vision and deep learning, researchers have developed automated systems for detecting disease in plant leaves that use images of the leaves to provide farmers with highly accurate and timely information about plant pest problems. While the majority of existing deep learning plant disease detection models are centralized training models that aggregate data obtained from multiple farms to a single server, this development creates several challenges for farmers and agricultural institutions. For example, farmers/farming institutions have many concerns regarding privacy, ownership, and communications costs associated with moving data to/from the remote server, and therefore may be uncertain about sharing raw image data with organizations that have no rights or ownership to the data collected.

The emergence of Federated Learning (FL) provides a way to address these challenges by introducing a decentralized framework for collaborative model training while limiting the amount of data that is shared with others. With FL, each of the participating clients (e.g., farms, agricultural research stations, or edge devices) trains its local model on its own private dataset and then transfers only the updates to that local model to the central server where they are aggregated. This greatly reduces the potential for privacy concerns, as well as the amount of communication that needs to take place, making FL particularly attractive for use in smart agriculture applications, where there is often a combination of heterogeneous data and a lack of resources.

Despite these benefits, using FL in agricultural settings presents its own unique set of challenges. For example, data utilized within an agricultural setting is often non-independent and not identically distributed (non-IID). Because there are so many different varieties of crops grown on farms, each with their own specific environmental conditions, as well as significant varia-

tion among the prevalence of disease and the imaging devices used to capture images of plants, the ability to train an accurate model will be greatly influenced by the algorithm selected to perform federated learning.

The loss of crops due to plant disease affects production very badly; farmers are not capable enough to detect these diseases by themselves. Therefore one solution is that crops would be regularly checked by agriculture experts, but it is not feasible as it requires a large number of experts. We can use different AI/ML models to detect diseases. But collecting data from different farms that are miles apart is difficult. Therefore we used federated learning to solve this problem. With federated learning, we can train our model on edge devices present on the farm locally with local data. In this way, there is no need to transfer data frequently to servers, and no privacy issue as data is kept locally. Instead of sending data, in federated learning we send model weights, therefore there is no privacy issue. Figure 1 shows the Federated Learning methodology for smart agriculture. Figure 2 illustrates the history of Federated Learning over the years.

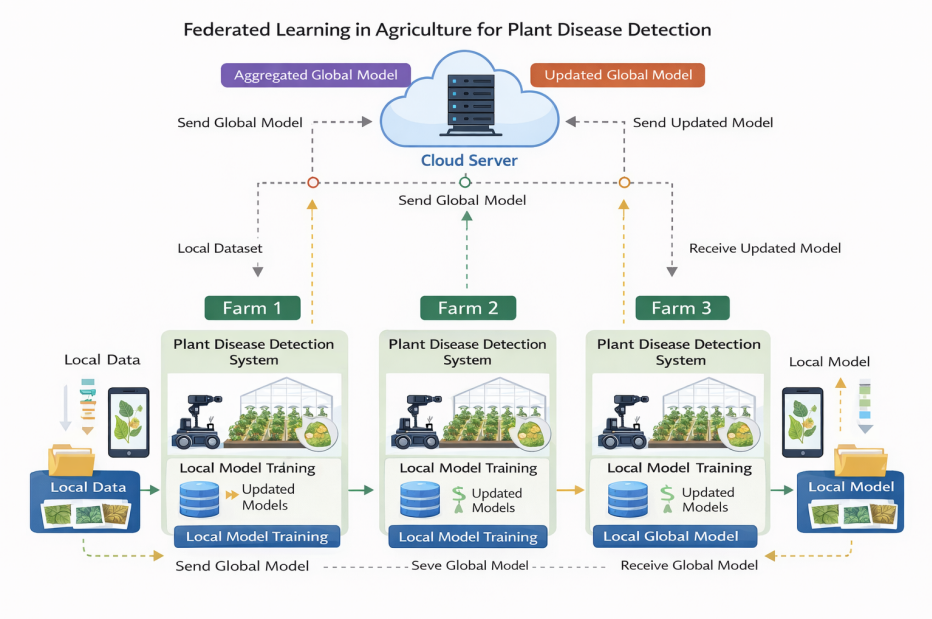


Figure 1: Federated Learning Methodology for Smart Agriculture

In this modern era, data is one of the most valuable assets. In other data analysis techniques such as machine learning and deep learning, farmers

have to send their farm data to servers in order to detect plant diseases. But farmers do not know how these corporations handle the data. Therefore, to protect privacy, federated learning techniques are used. In federated learning, instead of sending raw data to servers, models are trained on edge devices present on farmland; only the weights of the models are sent to the server, and the server aggregates the weights received from various farmers. In this way, there is no data privacy issue.

Apart from the privacy issue, it requires high-speed internet connectivity to send raw data rapidly. By sending only weights, the size of data decreases significantly. The baseline federated learning algorithm FedAvg completes multiple epochs of training on edge devices before sending to the server, which reduces the number of communication rounds significantly.

In this research paper, we have examined 5 federated learning algorithms with 5 different datasets which are explained in detail in subsequent sections.

2. Background

The fundamental objective of Federated Learning (FL) in agricultural phytopathology is to minimize a global objective function $f(w)$ by aggregating parameters from K distributed nodes (e.g., individual farms or edge servers). The global optimization problem is defined as:

$$\min_{w \in \mathbb{R}^d} f(w) = \sum_{k=1}^{K} p_k F_k(w) \tag{1}$$

where $p_k \geq 0$ and $\sum_k p_k = 1$. The term p_k typically represents the relative impact of the k-th client, often defined as n_k/n.

2.1. Federated Averaging (FedAvg)

FedAvg [1] is the baseline algorithm for federated learning. During each round, a random subset of clients runs multiple epochs of Local Stochastic Gradient Descent (SGD) on their local data, and then sends the resulting weights to the central server, where these weights are averaged in a weighted manner according to the clients' sample sizes.

Notation and Setup

- K: Total number of participating clients.

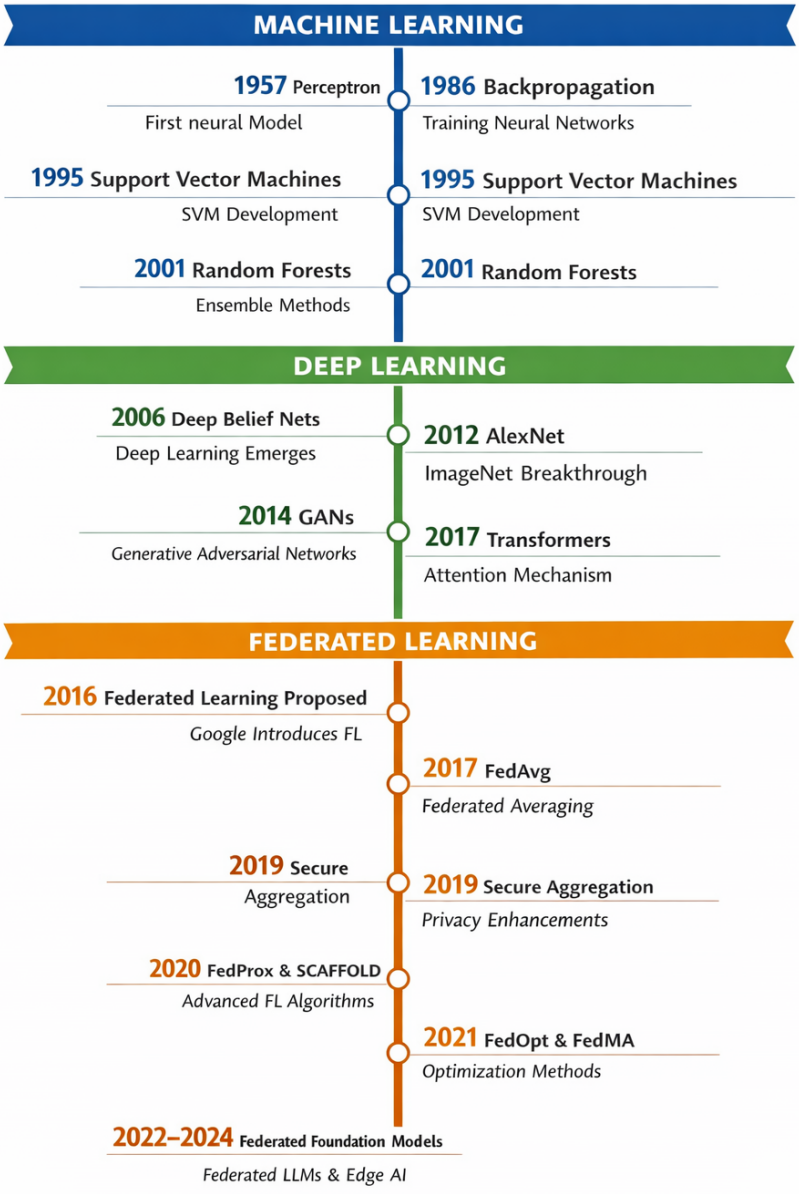


Figure 2: Emergence of Federated Learning

- k: Index of a specific client ($k = 1, 2, \ldots, K$).
- $\mathcal{D}_k$: The local dataset held by client k.
- n_k: The number of data samples on client k ($n_k = |\mathcal{D}_k|$).
- n: The total number of data samples across all clients ($n = \sum_{k=1}^{K} n_k$).
- w: The parameters (weights) of the machine learning model.
- $\ell(w; x, y)$: The loss function for a single data point (x, y) given model weights w.
- η: The learning rate for the local optimizer.
- E: The number of local epochs each client performs.

The Global Objective Function

In federated learning we calculate the weighted sum of model objective function $F_k(w)$:

$$\min_w f(w) = \sum_{k=1}^{K} \frac{n_k}{n} F_k(w) \tag{2}$$

Here, $F_k(w)$ represents the local objective function for client k, which is the average loss over its local dataset:

$$F_k(w) = \frac{1}{n_k} \sum_{(x_i, y_i) \in \mathcal{D}_k} \ell(w; x_i, y_i) \tag{3}$$

The FedAvg Algorithm (Round t)

FedAvg operates in discrete communication rounds. Let t be the current communication round.

Step a: Server Broadcast

At the beginning of round t, the central server selects a subset of clients S_t (where $|S_t| = C \times K$, and $C \in (0, 1]$ is the fraction of chosen clients). The server broadcasts the current global model weights w_t to these selected clients.

Step b: Local Client Update

Each selected client $k \in S_t$ initializes its local model with the global weights: $w_t^k = w_t$. The client then trains the model locally using SGD on its local dataset $\mathcal{D}_k$ for E epochs. The local weight update rule is:

$$w_{t,i+1}^k \leftarrow w_{t,i}^k - \eta \nabla F_k(w_{t,i}^k; \xi) \tag{4}$$

where ξ is a mini-batch sampled from $\mathcal{D}_k$, and i represents the local step index. After completing E epochs, the client obtains its new optimized local weights w_{t+1}^k.

Step c: Server Aggregation

The selected clients send their updated local weights w_{t+1}^k back to the central server. The server then computes the new global model w_{t+1} by taking a weighted average:

$$w_{t+1} = \sum_{k \in S_t} \frac{n_k}{n_{S_t}} w_{t+1}^k \tag{5}$$

where $n_{S_t} = \sum_{k \in S_t} n_k$ is the total number of data samples among the participating clients in round t.

FedAvg follows a synchronized communication protocol where each selected client k performs E local epochs of SGD on its private local dataset:

$$w_{t+1}^k = w_t - \eta \nabla F_k(w_t) \tag{6}$$

Upon completion, the server aggregates these updates:

$$w_{t+1} = \sum_{k=1}^{K} \frac{n_k}{n} w_{t+1}^k \tag{7}$$

While FedAvg is very efficient, it assumes data is IID, whereas in real life data is distributed in a heterogeneous nature.

2.2. Federated Proximal (FedProx)

In general, different devices have different types of data and different amounts of data, known as statistical heterogeneity. In addition, FL networks are prone to system heterogeneity. For example, some devices are slow (stragglers) whereas others are fast. In FedAvg, if straggler devices fail to

complete the required number of epochs, they are dropped from the training round, which could negatively affect convergence.

FedProx [2] tackles the statistical and system heterogeneity of FL by introducing a proximal term to the local objective. This term penalizes the local update if it goes too far from the current global model.

The FedProx Algorithm (Round t)

Step a: Server Broadcast

Just like FedAvg, the central server selects a subset of clients S_t and broadcasts the current global model weights w_t.

Step b: Local Client Update

Each selected client $k \in S_t$ initializes its local model $w_t^k = w_t$. The client then applies SGD to its modified objective function. The local gradient update step becomes:

$$w_{t,i+1}^k \leftarrow w_{t,i}^k - \eta \left(\nabla F_k(w_{t,i}^k; \xi) + \mu(w_{t,i}^k - w_t) \right) \tag{8}$$

Because FedProx is designed to handle systems heterogeneity, client k performs γ_k epochs of work based on its available computing resources, resulting in the locally updated model w_{t+1}^k.

Step c: Server Aggregation

The server aggregates the local models from the participating clients:

$$w_{t+1} = \sum_{k \in S_t} \frac{n_k}{n_{S_t}} w_{t+1}^k \tag{9}$$

2.3. Stochastic Controlled Averaging (SCAFFOLD)

SCAFFOLD [3] addresses the client drift issue by using control variates. Control variates are estimations for update direction for the server model (c) and for each client (c_k). The difference ($c - c_k$) is the estimation of client drift which is used for correcting local updates. Due to sending extra parameters, network load increases, and therefore SCAFFOLD is feasible only with high bandwidth.

Notations

- c: The global control variate, representing the overall gradient direction.
- c_k: The local control variate for client k.
- η_l: The local learning rate.
- η_g: The global learning rate at the server level.
- E: The number of local steps performed by the client.

The SCAFFOLD Algorithm (Round t)

Step a: Server Broadcast

The central server selects a subset of clients S_t and broadcasts both the current global model weights w_t and the global control variate c_t.

Step b: Local Client Update

Each selected client $k \in S_t$ initializes $w_{t,0}^k = w_t$. During local training, the client modifies its SGD update rule:

$$w_{t,i+1}^k = w_{t,i}^k - \eta_l \left(\nabla F_k(w_{t,i}^k; \xi) - c_k + c_t \right) \tag{10}$$

The term $(-c_k + c_t)$ acts as a gradient correction factor.

Step c: Local Control Variate Update

Before sending data back, client k updates its own local control variate:

$$c_k^+ = c_k - c_t + \frac{1}{E\eta_l}(w_t - w_{t,E}^k) \tag{11}$$

The client computes the deltas to send to the server:

- Model delta: $\Delta w^k = w_{t,E}^k - w_t$
- Control variate delta: $\Delta c^k = c_k^+ - c_k$

Step d: Server Aggregation

$$w_{t+1} = w_t + \frac{\eta_g}{|S_t|} \sum_{k \in S_t} \Delta w^k \tag{12}$$

$$c_{t+1} = c_t + \frac{1}{|S_t|} \sum_{k \in S_t} \Delta c^k \tag{13}$$

This ensures that even if a specific node contains images or data of only one particular type, its updates remain aligned with the broader global objective.

2.4. Federated Matched Averaging (FedMA)

FedMA [4] stands for Federated Matched Averaging. Instead of directly averaging weights, it first matches neurons using the Hungarian algorithm and then applies averaging on matched neurons. It processes the network layer by layer; by matching similar features from different client models, FedMA obtains a more compact global model.

Layer-wise Procedure for FedMA
Input: Local model weights from J clients, each with L layers: $\{W_1^{(j)}, W_2^{(j)}, \ldots, W_L^{(j)}\}_{j=1}^{J}$.
Output: Global model weights: $\{W_1^{\text{global}}, W_2^{\text{global}}, \ldots, W_L^{\text{global}}\}$.

1. **Initialize:** Collect all locally trained client models at the central server.
2. **Iterate over hidden layers:** For each layer $l = 1, 2, \ldots, L-1$:
   (a) **Solve Matching Problem:** The server computes the optimal permutation matrices $\Pi_l^{(j)}$ for each client j using the Hungarian algorithm:

$$\min_{\{\Pi_l^{(j)}\}, \theta_l} \sum_{j=1}^{J} \text{cost}\left(\Pi_l^{(j)} W_l^{(j)}, \theta_l\right) \tag{14}$$

   (b) **Aggregate Current Layer:**

$$W_l^{\text{global}} = \frac{1}{J} \sum_{j=1}^{J} \Pi_l^{(j)} W_l^{(j)} \tag{15}$$

   (c) **Adapt Next Layer Weights:**

$$W_{l+1}^{(j)} \leftarrow W_{l+1}^{(j)} \left(\Pi_l^{(j)}\right)^T \quad \forall\, j \in \{1, \ldots, J\} \tag{16}$$

3. **Final Layer Aggregation:** The output layer L does not require matching because the classes are already aligned across all clients:

$$W_L^{\text{global}} = \frac{1}{J} \sum_{j=1}^{J} W_L^{(j)} \tag{17}$$

2.5. Adaptive Federated Optimization (FedOpt)

FedOpt [5] integrates adaptive optimization principles—such as Adam or Adagrad—at the server side. The server treats the difference between the global model and the average of client updates as a "pseudo-gradient" Δ_t:

$$\Delta_t = w_t - \sum_{k=1}^{K} \frac{n_k}{n} w_{t+1}^k \tag{18}$$

The update uses an adaptive second moment estimate v_t to scale the learning rate:

$$w_{t+1} = w_t - \eta_s \frac{\Delta_t}{\sqrt{v_t} + \epsilon} \tag{19}$$

This allows for faster convergence and higher accuracy, particularly when the global model must adapt to emerging plant pathogens across a large network.

3. Literature Study

We have studied several plant disease datasets openly available for our current study. Table 1 shows the dataset details.

Table 2 shows the literature study related to the application of Federated Learning algorithms in smart agriculture.

Table 1: Summary of Agricultural Datasets for Plant Disease Detection

S.No	Name	No. of Images	Classes	Description	Citation
1	PlantVillage	54,306	38	14 crop species (Grape, Apple, Corn, Tomato, etc.) with healthy and diseased labels, laboratory conditions.	[6]
2	Tomato Leaf Disease	11,000	10	Tomato leaf images covering Bacterial Spot, Early Blight, Late Blight, Leaf Mold, Mosaic Virus, Septoria Leaf Spot, Spider Mites, Target Spot, Yellow Leaf Curl Virus, and Healthy.	[7]
3	Wheat Plant Diseases	14,000+	Multiple	Wheat crop disease images for classification of various wheat diseases; collected under field conditions.	[8]
4	Cotton Plant Disease	∼1,951	4	Field-captured cotton leaf images covering Bacterial Blight, Curl Virus, Fussarium Wilt, and Healthy classes.	[9]
5	Potato Plant Diseases	∼2,152	3	Potato leaf images with Early Blight, Late Blight, and Healthy classes; collected for federated training experiments.	[10]

Table 2: Literature Review of Federated Learning Applications in Agricultural Disease Detection

S.No	Year	Proposed Method	Dataset	Performance	Ref.
1	2023	FL with ResNet50, MobileNetV2, VGG16, DenseNet121, InceptionV3, ViT-B16, ViT-B32 (FedAvg); best model: ResNet50	PlantVillage (38 classes; >50,000 images)	ResNet50: Acc 99.59%, F1 99.59% (3 clients, 30 rounds); MobileNetV2: Acc 98.58%; VGG16: Acc 98.68%; DenseNet121: Acc 98.39%	[11]
2	2025	FL with VGG16 and cloud computing infrastructure; FedAvg aggregation across distributed agricultural nodes	PlantVillage (plant leaf disease images)	Acc: 99.71% (VGG16 after FL aggregation)	[12]
3	2024	Federated Deep Learning with Intelligent Weight Transferring (FDL-IWT); adaptive knowledge transfer across FL clients	Apple, Maize, Tomato disease datasets (PlantVillage)	Testing Acc: 97.5% (highest among 7 compared methods)	[13]

Continued on next page

Table 2 continued from previous page

S.No	Year	Proposed Method	Dataset	Performance	Ref.
4	2023	Federated Transfer Learning (F-TL) and Federated Feature Extraction (F-FE) using EfficientNetB3; resource-efficient FL over IoAT edge devices	Custom rice leaf disease dataset (5,932 images; bacterial leaf blight, blast, brown spot, tungro)	F-TL Val. Acc: 99%; F-FE: reduced GPU memory & CPU utilization; both methods Acc: 99%	[14]
5	2025	Lightweight Vision Transformer (ViT) with attention mechanism in FL setting; multi-agent federated framework	Kaggle Mango Leaf Disease dataset	Acc: 98.99% (FL scenario, lightweight ViT)	[15]
6	2024	FL with five CNNs (VGG-11, AlexNet, ResNet-18, SqueezeNet, ShuffleNet) under distributed paradigm using FedLab; FedAvg aggregation	PlantVillage – Maize subset (common rust 1192, gray leaf spot 513, northern leaf blight 985, healthy 1162 images)	VGG-11: Acc 97.29 ± 0.18%, F1 96.24±0.27%; AlexNet: Acc 96.87 ± 0.34%; ShuffleNet: Acc 75.65 ± 2.00% (lowest)	[16]
7	2026	FL + Transfer Learning (FL-TL) for maize leaf classification; optimized federated aggregation for IID and non-IID data	Maize leaf disease dataset (IID and non-IID splits)	FL+TL on non-IID: Acc 99.89%	[17]

Continued on next page

Table 2 continued from previous page

S.No	Year	Proposed Method	Dataset	Performance	Ref.
8	2025	Decentralized FL (DFL) using Loss_{val} for peer-to-peer model sharing and local learning correction; ResNet50, VGG16, ViT_B16	PlantVillage (IID and non-IID; 6 and 18 clients)	ResNet50: best at λ=0.75; VGG16: best at λ=0.25 (highest stability); Acc, Precision, Recall, F1 all reported; DFL outperforms centralized FL in convergence & generalization	[18]
9	2025	AGRIFOLD: agriculture-specific FL framework with optimized federated training for multi-client leaf disease detection	Multi-crop leaf disease dataset (PlantVillage-based)	Training Acc: 99.18%; Test Acc: 99.06%; F1-score systematically evaluated	[19]
10	2025	Review of FL privacy-preserving techniques (Differential Privacy, Secure Aggregation, Homomorphic Encryption) applied to smart farming including disease detection	Multi-source agricultural datasets reviewed across FL architectures	Review paper: benchmarks FL for disease detection, yield prediction, and resource management across distributed agricultural clients	[20]

4. Experimental Setup

4.1. Datasets and Data Distribution

Five publicly available agricultural image datasets were used in this study, namely PlantVillage [6], Tomato Leaf Disease [7], Wheat Plant Diseases [8], Cotton Plant Disease [9], and Potato Plant Diseases [10]. for simulating the realistic federated learning environment, the datasets were distributed among multiple clients representing geographically separated farms.

A non-IID data distribution strategy was adopted because agricultural data collected from different farms generally exhibit heterogeneous characteristics due to variations in crop species, disease prevalence, environmental conditions, and image acquisition devices.

The datasets were partitioned among five clients using a Dirichlet distribution with concentration parameter $\alpha = 0.5$. A lower value of α generates stronger statistical heterogeneity and better represents practical agricultural scenarios where individual farms may contain only a subset of disease classes.

Each client retained its local data throughout training and only model parameters were exchanged with the central server.

4.2. Model Architecture

A ResNet18 convolutional neural network architecture was employed as the base classification model for all federated learning algorithms. ResNet18 was selected because of its balance between computational efficiency and classification performance in image recognition tasks. The final fully connected layer was modified according to the number of disease classes present in each dataset. The same neural network architecture was used for all federated learning algorithms to ensure a fair comparison.

4.3. Training Parameters

To ensure fair comparison, identical hyperparameters were used across all federated learning algorithms. The Stochastic Gradient Descent (SGD) was used as local optimizer with momentum 0.9. The learning rate was fixed 0.001. The batch size was set to 32. Each client performed five local epochs before communicating model updates to the central server. The global training process was executed for 50 communication rounds.

4.4. Federated Learning Settings

Five federated learning algorithms were evaluated: FedAvg, FedProx, SCAFFOLD, FedMA, and FedOpt. In FedProx, the proximal coefficient μ was fixed at 0.01. For SCAFFOLD, both global and local control variates were maintained throughout training. FedMA utilized neuron matching through the Hungarian algorithm before aggregation. FedOpt employed the Adam optimizer at the server side with learning rate 0.001.

4.5. Evaluation Metrics

The performance of the federated learning models was evaluated using Accuracy, Precision, Recall, and F1-score. The metrics are computed as:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{20}$$

$$\text{Precision} = \frac{TP}{TP + FP} \tag{21}$$

$$\text{Recall} = \frac{TP}{TP + FN} \tag{22}$$

$$\text{F1} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{23}$$

4.6. Hardware Configuration

The experiments were conducted using the Kaggle cloud computing platform equipped with NVIDIA T4 x2 GPUs, 16 GB GPU memory, and approximately 30 GB RAM. Python 3.11 was used as the programming language along with PyTorch libraries for deep learning implementation.

5. Results and Discussion

In this section, we present the empirical results of our comparative study. Figure 3 shows the proposed approach for Federated Learning based methods for our current scenario. Table 4 illustrates the performance metrics across all five datasets for each federated learning algorithm.

The primary objective of this study is to evaluate the efficacy of five prominent FL optimization algorithms—FedAvg, FedProx, SCAFFOLD, FedMA, and FedOpt—across five distinct agricultural image datasets: PlantVillage, Tomato Leaf, Wheat Plant, Cotton Plant, and Potato Plant.

Table 3: Experimental Hyperparameters

Parameter	**Value**
Number of Clients	5
Communication Rounds	50
Local Epochs	5
Batch Size	32
Learning Rate	0.001
Optimizer	SGD
Momentum	0.9
Non-IID Split	Dirichlet ($\alpha = 0.5$)
FedProx μ	0.01
FedOpt Optimizer	Adam

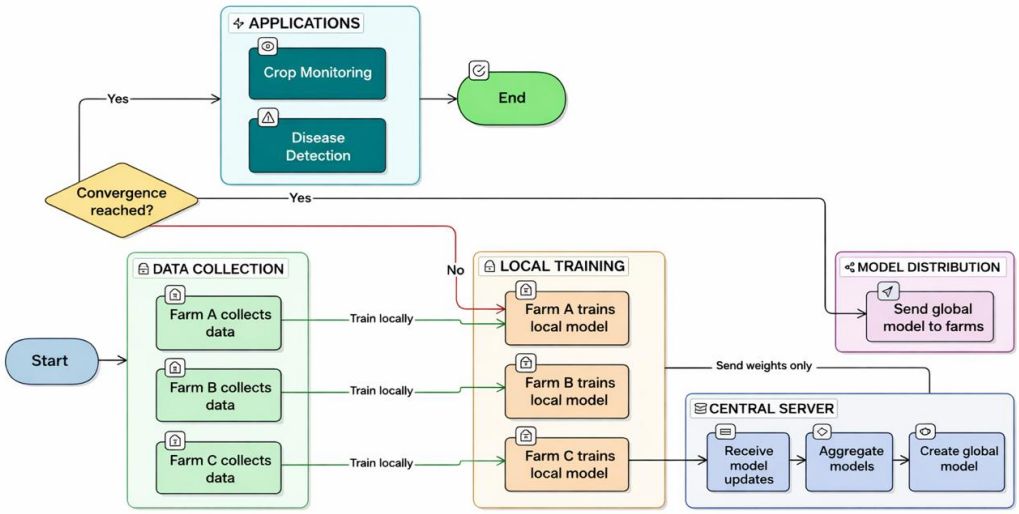


Figure 3: Proposed Comparative Study on different datasets

FedProx, FedMA, and SCAFFOLD show very low precision compared to FedAvg and FedOpt, generating more false positives. The recall values for all algorithms are similar, indicating that every model identifies diseases with comparable effectiveness. FedOpt handles non-IID data better than others because it uses server-side adaptive optimization (Adam) which reduces client drift and stabilizes convergence. SCAFFOLD and FedMA show moderate performance, while FedProx performs slightly lower, especially in Precision and F1-score. Overall, FedOpt performed best.

Table 4: Performance comparison (Accuracy, Precision, Recall, F1-Score) across all algorithms and datasets

Algorithm	**Cotton Plant**	**Wheat Plant**	**Potato Plant**	**Tomato Leaf**	**PlantVillage**
FedAvg	A: 0.7387 P: 0.7296 R: 0.7387 F: 0.7324	A: 0.8456 P: 0.8422 R: 0.8456 F: 0.8416	A: 0.9838 P: 0.9843 R: 0.9838 F: 0.9838	A: 0.9927 P: 0.9927 R: 0.9927 F: 0.9926	A: 0.9913 P: 0.9891 R: 0.9863 F: 0.9876
FedProx	A: 0.7376 P: 0.7347 R: 0.7376 F: 0.7345	A: 0.8452 P: 0.8415 R: 0.8452 F: 0.8409	A: 0.9884 P: 0.9887 R: 0.9884 F: 0.9884	A: 0.9837 P: 0.9825 R: 0.9840 F: 0.9836	A: 0.9936 P: 0.9936 R: 0.9936 F: 0.9935
SCAFFOLD	A: 0.6458 P: 0.7177 R: 0.6458 F: 0.6338	A: 0.2167 P: 0.1898 R: 0.2167 F: 0.1946	A: 0.4644[†] P: 0.2157 R: 0.4644 F: 0.2945	A: 0.5260 P: 0.6750 R: 0.5260 F: 0.4950	A: 0.0481[†] P: 0.0023 R: 0.0481 F: 0.0044
FedMA	A: 0.7650 P: 0.8073 R: 0.7650 F: 0.7612	A: 0.9861 P: 0.9865 R: 0.9861 F: 0.9857	A: 0.9861 P: 0.9865 R: 0.9861 F: 0.9857	A: 0.9837 P: 0.9840 R: 0.9837 F: 0.9836	A: 0.9963 P: 0.9954 R: 0.9934 F: 0.9943
FedOpt	A: 0.4768 P: 0.2960 R: 0.2650 F: 0.2534	A: 0.9194 P: 0.8460 R: 0.9194 F: 0.8812	A: 0.9186 P: 0.8580 R: 0.9186 F: 0.8856	A: 0.1000[‡] P: 0.1550 R: 0.1000 F: 0.0300	A: 0.4709 P: 0.6061 R: 0.4709 F: 0.4256

A = Accuracy, P = Precision, R = Recall, F = F1-score.

[†] SCAFFOLD produced a device mismatch runtime error due to tensors residing on different devices (CPU/GPU) during aggregation; results will be updated after re-run.

[‡] FedOpt (FedAdam, ServerLR = 0.01) diverged on Tomato Leaf. Accuracy decreased from 0.335 (Round 1) to 0.100 (Round 10).

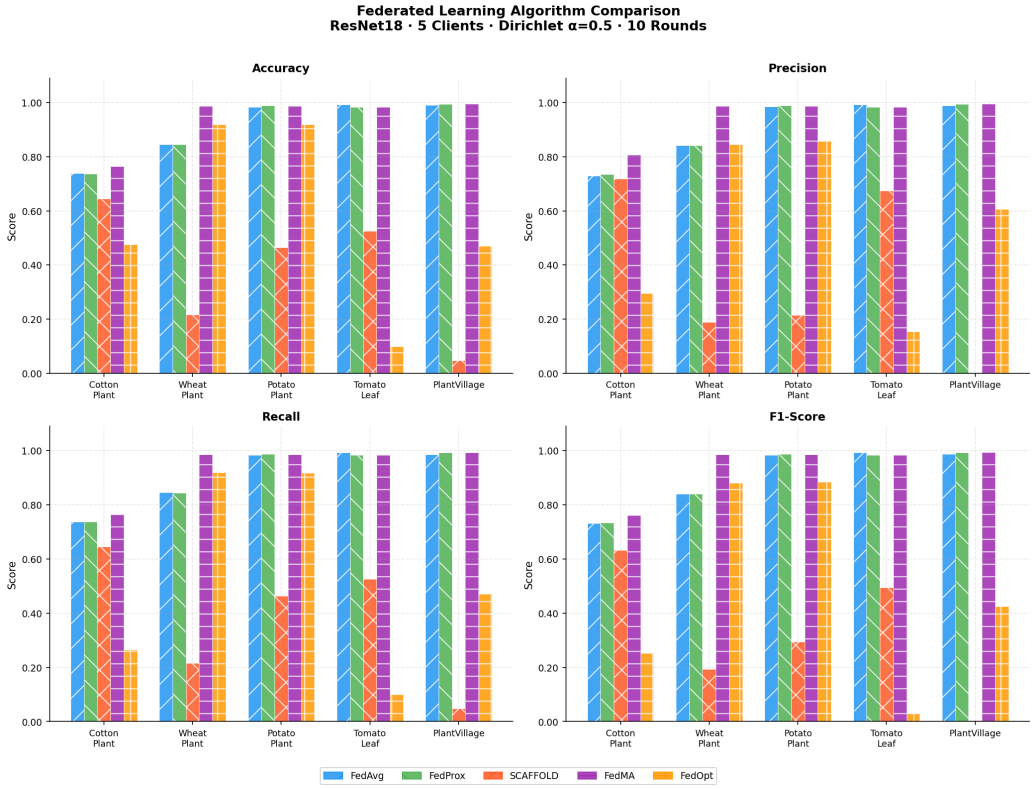


Figure 4: Comparison of Federated Learning algorithms on different datasets

6. Conclusion

This paper has provided a thorough comparative analysis of five state-of-the-art federated learning algorithms, namely FedAvg, FedProx, SCAFFOLD, FedMA, and FedOpt, for image-based plant disease diagnosis in a realistic, non-IID setting representative of geographically distributed agricultural environments. Through the evaluation of these algorithms on different CNN architectures and the measurement of their performance using accuracy, precision, recall, F1-score, convergence speed, and communication cost, this paper has provided a holistic insight into their applicability in smart agriculture.

In general, the results emphasize that there is no FL algorithm that is optimal in all scenarios; instead, the choice of algorithm depends on the level of data heterogeneity and communication constraints available at the client

level. This comparative study provides useful insights into the selection of suitable FL approaches for plant disease detection systems while maintaining data privacy. Future research will concentrate on combining adaptive client selection, lightweight models, and multi-modal data to improve robustness, scalability, and real-time performance in precision agriculture.

Declarations

- **Funding:** Not applicable.
- **Competing interests:** The authors declare no competing interests.
- **Ethics approval:** All authors have read, understood, and have complied as applicable with the statement on "Ethical responsibilities of Authors" as found in the Instructions for Authors.
- **Consent to publish:** Agreed.
- **Clinical trial number:** Not applicable.
- **Data availability:** No datasets were generated in the presented study. All public datasets used in the study have been cited in the references.
- **Author contribution:** All authors contributed equally in preparing the manuscript. Jaspal Kaur Saini reviewed and revised the final manuscript.

Declaration of Competing Interest

The authors declare no competing interests.

Data Availability

Data associated with the manuscript are available from the corresponding author on reasonable request.

References

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, B. A. y Arcas, Communication-efficient learning of deep networks from decentralized data, in: Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS), Vol. 54, PMLR, 2017, pp. 1273–1282, jMLR W&CP Vol. 54. doi:10.48550/arXiv.1602.05629.

[2] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, V. Smith, Federated optimization in heterogeneous networks, Proceedings of Machine Learning and Systems 2 (2020) 429–450.
URL https://mlsys.org/virtual/2020/poster/1343

[3] S. P. Karimireddy, S. Kale, M. Mohri, S. J. Reddi, S. U. Stich, A. T. Suresh, SCAFFOLD: Stochastic controlled averaging for federated learning, in: Proceedings of the 37th International Conference on Machine Learning (ICML), Vol. 119, PMLR, 2020, pp. 5132–5143. doi:10.48550/arXiv.1910.06378.

[4] H. Wang, M. Yurochkin, Y. Sun, D. Papailiopoulos, Y. Khazaeni, Federated learning with matched averaging, in: Proceedings of the 8th International Conference on Learning Representations (ICLR), 2020.
URL https://openreview.net/forum?id=BkluqlSFDS

[5] S. J. Reddi, Z. Charles, M. Zaheer, Z. Garrett, K. Rush, J. Konečný, S. Kumar, H. B. McMahan, Adaptive federated optimization, in: Proceedings of the 9th International Conference on Learning Representations (ICLR), 2021.
URL https://openreview.net/forum?id=LkFG3lB13U5

[6] D. P. Hughes, M. Salathé, An open access repository of images on plant health to enable the development of mobile disease diagnostics, arXiv preprint abs/1511.08060 (2015). doi:10.48550/arXiv.1511.08060.
URL https://arxiv.org/abs/1511.08060

[7] K. Bhalerao, Tomato leaf disease detection dataset, Kaggle, 10 classes, 11,000 images (256×256 RGB). https://www.kaggle.com/datasets/kaustubhb999/tomatoleaf (2020).

[8] K. Agarwal, Wheat plant diseases dataset, Kaggle, 14,000+ images for wheat disease classification. https://www.kaggle.com/datasets/kushagra3204/wheat-plant-diseases (2024).

[9] Dhamur, Cotton plant disease dataset, Kaggle, ∼1,951 images; 4 classes (Bacterial Blight, Curl Virus, Fussarium Wilt, Healthy). https://www.kaggle.com/datasets/dhamur/cotton-plant-disease (2021).

[10] H. Nouman, Potato plant diseases data, Kaggle, ∼2,152 images; 3 classes (Early Blight, Late Blight, Healthy). https://www.kaggle.com/datasets/hafiznouman786/potato-plant-diseases-data (2022).

[11] D. Mamba Kabala, A. Hafiane, L. Bobelin, R. Canals, Image-based crop disease detection with federated learning, Scientific Reports 13 (1) (2023) 19220. doi:10.1038/s41598-023-46218-5.

[12] C. S. G. da Silva, A. M. Neto, et al., Plant disease detection using federated learning and cloud infrastructure for scalability and data privacy, Journal of Internet Services and Applications 16 (1) (2025) 530–543. doi:10.5753/jisa.2025.5055.

[13] P. Hari, M. P. Singh, Adaptive knowledge transfer using federated deep learning for plant disease detection, Computers and Electronics in Agriculture 229 (2025) 109720. doi:10.1016/j.compag.2024.109720.

[14] M. Aggarwal, V. Khullar, N. Goyal, R. Gautam, F. Alblehai, M. Elghatwary, A. Singh, Federated transfer learning for rice-leaf disease classification across multiclient cross-silo datasets, Agronomy 13 (10) (2023) 2483. doi:10.3390/agronomy13102483.

[15] V. Gautam, P. Kaur, A. M. Mishra, A. Shankar, L. Singh, Mango leaf disease detection and classification using a federated learning based lightweight vision transformer model, Turkish Journal of Agriculture and Forestry 49 (6) (2025) 1079–1094. doi:10.55730/1300-011X.3325.

[16] T. M. Antico, L. F. Rodrigues Moreira, R. Moreira, Evaluating federated learning for maize leaf disease classification using convolutional neural networks, Biosystems Engineering 251 (2025) 1–14. doi:10.1016/j.biosystemseng.2025.01.021.

[17] Author(s), Optimizing federated learning for maize leaf disease classification using transfer learning under IID and non-IID data distributions, Cluster Computing 29 (2) (2026). doi:10.1007/s10586-025-05888-8.

[18] D. Mamba Kabala, A. Hafiane, L. Bobelin, R. Canals, Decentralized federated learning using validation loss for model sharing and local learning correction in crop disease classification, Ecological Informatics 87 (2025) 103083. doi:10.1016/j.ecoinf.2025.103083.

[19] F. Piccialli, C. Della Bruna, D. Chiaro, P. Qi, M. Savoia, AGRIFOLD: AGRiculture federated learning for optimized leaf disease detection, Expert Systems with Applications 289 (2025) 128371. doi:10.1016/j.eswa.2025.128371.

[20] R. Dembani, I. Karvelas, N. A. Akbar, S. Rizou, D. Tegolo, S. Fountas, Agricultural data privacy and federated learning: A review of challenges and opportunities, Computers and Electronics in Agriculture 232 (2025) 110048. doi:10.1016/j.compag.2025.110048.